Semantic Web
XX(X):1–19

DOI: 10.1177/ToBeAssigned
www.sagepub.com/
SAGE

DAIMO: An Ontology Profile for Governed AI Model Assets in Dataspaces

Edmundo de Elvira Mori Orrillo[1] and Jiayun Liu[1]

Abstract

Inter-organisational deployment of artificial-intelligence models requires a coherent semantic representation of five tasks that remain fragmented across distinct vocabularies: publishing a model as a catalogue asset, selecting it under a usage policy, invoking it through a typed service, tracing each execution, and comparing models under reproducible evaluation conditions. MLDCAT-AP describes the model; DCAT publishes it; ODRL expresses policy; PROV-O records provenance; and the *Dataspace Protocol* (DSP) negotiates contractual exchange. No single vocabulary articulates these tasks as one governance chain.

This article presents DAIMO (*Dataspace AI Model Ontology*), an OWL 2 DL integration profile that reuses the vocabularies above through verified axiomatic alignments. DAIMO introduces fourteen native classes (nine top-level classes plus five role subclasses) to cover bridge relations not expressed directly by the reused vocabularies, and is accompanied by a SHACL layer with structural constraints, six SHACL-SPARQL cross-class invariants, and a negative test bench.

DAIMO was developed with the *Linked Open Terms* (LOT) methodology and is exercised over a multi-participant demonstrator that instantiates the twenty-three competency questions formulated during requirements specification. The evaluation reports OWL consistency, OWL-RL materialisation, SHACL conformance, negative tests, SPARQL queries, and a bounded synthetic scalability benchmark.

Keywords

AI model governance, dataspace, ontology alignment, SHACL validation, LOT methodology

Introduction

Regulation (EU) 2024/1689, the *AI Act*, requires technical documentation, execution records, lifecycle traceability, and audit evidence for high-risk AI systems[1]. At the same time, European dataspaces promote contractual exchange between autonomous participants through catalogues, policies, and protocols such as the *Dataspace Protocol* (DSP)[3], implemented in connectors such as Eclipse Dataspace Components (EDC)[2]. In this setting, an AI model is not only a technical artefact: it is a governed asset that must be published, discovered, invoked, traced, and evaluated across organisational boundaries.

Existing vocabularies cover important fragments of that lifecycle. MLDCAT-AP 3.0.0[4] extends DCAT to describe machine-learning models as catalogue assets, with metadata on tasks, runs, evaluations, risks, and compute resources. DCAT[7] provides the catalogue and service layer; ODRL[8] represents offers, permissions, prohibitions, and agreements; PROV-O[9] records provenance over entities, activities, and agents; and DSP specifies contractual exchange in dataspaces. Model cards[10], factsheets[11], and datasheets[12] complement these vocabularies with narrative documentation.

The gap identified in the reviewed literature is relational. A model described with MLDCAT-AP is not, by that fact alone, connected to the act of being offered in a federated catalogue. An ODRL policy can express permissions without being tied to the deployment it governs. A DSP negotiation does not necessarily make explicit the evaluation context or audit evidence associated with an AI invocation. A PROV bundle can group provenance without distinguishing the chain that crosses several administrative contexts. The problem is therefore not the lack of another model description, but the need to articulate catalogue, policy, deployment, execution, evidence, and evaluation as one queryable and validatable chain.

This article presents DAIMO (*Dataspace AI Model Ontology*), an OWL 2 DL integration profile for governed AI models in dataspaces. DAIMO follows a reuse-first strategy: it preserves `it6:MachineLearningModel` as the model class from MLDCAT-AP and uses DCAT, ODRL, PROV-O, DSP/EDC, FOAF, SKOS, and SPDX wherever they already provide the required semantics. It introduces bridge classes only when the relation between vocabularies is not otherwise covered.

The contributions are organised in three parts:

1. a reuse-first integration profile with fourteen native classes, of which nine are top-level classes and five

[1]Department of Artificial Intelligence, ETSI Informáticos, Universidad Politécnica de Madrid, Spain

Corresponding author:
Edmundo de Elvira Mori Orrillo, Department of Artificial Intelligence, ETSI Informáticos, Universidad Politécnica de Madrid, Spain.
Email: edmundo.mori.orrillo@upm.es

are role subclasses, oriented to bridge relations between catalogue, policy, deployment, execution, provenance, and evaluation;

2. reproducible validation of the ontology artefact through OWL consistency, OWL-RL materialisation, entailment verification, SHACL conformance, a negative invariant test bench, an OOPS! scan, twenty-three competency queries, and a bounded synthetic scalability benchmark;

3. a multi-participant demonstrator, deliberately synthetic, that instantiates the profile over a risk-prediction scenario and supports representational adequacy assessment without being presented as production deployment or external expert validation.

The reported evidence is therefore executable evidence over the ontology artefact and over a demonstration graph. It is not interpreted as legal certification, institutional adoption, or validation of a federated production platform. The remainder of the article reviews related work, describes methodology and requirements, presents the ontology, reports evaluation, discusses limits and evolution paths, and summarises artefact availability.

Related work

DAIMO sits at the intersection of four families of work that the existing literature usually treats separately: narrative model documentation, machine-learning lifecycle ontologies, governance vocabularies, and dataspace standards. We review each family in order of proximity to the problem, identifying what it contributes to the catalogue-dataspace bridge, where it leaves the problem open, and how DAIMO is positioned with respect to it. A fifth subsection reviews recent *Semantic Web Journal* ontologies whose validation methodology we adopt, before closing the section with a reuse matrix inspired by RePlanIT[28].

Describing AI models

Narrative documentation. Model cards[10], factsheets[11], and datasheets for datasets[12] raised transparency standards for AI artefacts. They describe intended use, limitations, expected benchmark behaviour, and training conditions in natural language. They are valuable for human communication and qualitative accountability, but insufficient as a substrate for automated discovery, policy-based filtering, or evidence linking in machine-to-machine flows: a dataspace requires primitives that software agents can consume, not documents that other agents must interpret.

ML lifecycle ontologies. Where narrative documentation fails for machine consumption, structured ML lifecycle ontologies take its place. EXPO[13] offers a general ontology of scientific experiments that later work has built on. ML-Schema[14] formalises machine-learning experiments as a reusable, lightweight schema. MEX[15] provides an interchange vocabulary, and OntoDM[16] together with DMOP[17] organise the data-mining domain. Souza et al.[21] and Schmidt et al.[22] extend the perspective towards provenance and FAIRness across the lifecycle. These ontologies provide a rigorous foundation but remain centred on the scientific experiment and the training process; none addresses the contractual dimension of exchange between autonomous participants.

A recent group of works shifts from the experiment to the taxonomy and management of AI assets. The *Artificial Intelligence Ontology* (AIO) systematises technical and ethical AI concepts through branches such as bias, layers, machine-learning tasks, mathematical functions, models, networks, preprocessing, and training strategies[18]. The *Intelligence Task Ontology* (ITO) organises tasks, benchmarks, and performance metrics to analyse AI capability progress at scale[19]. Novacek et al. propose ontology-supported management of models and datasets, focusing on the relation between models and the datasets used for training and validation[20]. These works reinforce the need to describe model structure, datasets, and metrics; DAIMO addresses a different and complementary question: how such descriptions become offerable, authorisable, invocable, auditable, and comparable assets inside a federated dataspace.

The closest and most recent standard-facing work is MLDCAT-AP 3.0.0[4], published by SEMIC in 2025. MLDCAT-AP extends DCAT to describe AI models as subclasses of `dcat:Dataset`, with rich metadata on policy, benchmarks, evaluation, execution, risk, and compute infrastructure. DAIMO reuses it directly: `it6:MachineLearningModel` is the model class in DAIMO, and `it6:Task`, `it6:Run`, `it6:Flow`, `it6:Evaluation`, and `it6:ComputerInfrastructure` capture respectively the target task, the concrete execution, the associated pipeline, the resulting measurement, and the compute environment. MLDCAT-AP provides strong coverage of model-side metadata but is deliberately agnostic about the dataspace in which the model circulates: it does not model the act of offering a model across organisational boundaries, nor governed deployment, execution authorisation anchored to an agreement, or provenance aggregated across distinct administrative contexts. MLDCAT-AP covers the model; DAIMO covers the bridge between the model and the dataspace.

Table 1 makes the class-by-class relation between DAIMO and MLDCAT-AP explicit. The pattern is intentional: DAIMO adopts MLDCAT-AP classes for the AI-asset plane, adds profile constraints when governed exchange requires operational completeness, and introduces extensions only for bridge relations that MLDCAT-AP does not cover. The evaluated profile version remains on MLDCAT-AP 3.0.0 because alignments, examples, and SHACL tests were built against that specification. MLDCAT-AP 3.1.0, published as a *Candidate Recommendation* in 2026[5], remains an evolution risk that future alignment versions will need to assess.

Governing resources and dataspaces

W3C governance vocabularies. ML lifecycle ontologies describe what a model is and how it was produced, but they do not govern how it is published, licensed, or attributed. Three W3C vocabularies provide that governance substrate. DCAT 2[7] provides the catalogue primitives on which both MLDCAT-AP and DAIMO rest. ODRL 2.2[8] offers a consolidated model for permissions, prohibitions, obligations, offers (`odrl:Offer`), and binding agreements

Table 1. Relationship between DAIMO and MLDCAT-AP 3.0.0.

Construct	DAIMO treatment	Notes
`it6:MachineLearningModel`	Adopted + profiled	AI asset description; DAIMO requires policy, title, and identifier for governed exchange.
`it6:Task`	Adopted	Target task of a model.
`it6:Run`	Adopted + profiled	Execution record; DAIMO requires flow, algorithm, agent, timestamp, and authorisation linkage.
`it6:Flow`	Adopted	Pipeline associated with a run.
`it6:Evaluation`	Adopted	Measurement result; can be included in cross-participant provenance.
`it6:EvaluationMeasure`	Adopted	Metric type and value.
`it6:Benchmark`	Adopted	Benchmark reference for comparability conditions.
`it6:ComputerInfrastructure`	Adopted	Compute environment on which a deployment runs.
`it6:Hardware`, `it6:Library`	Adopted	Elements of the compute environment.
`it6:HarmRisk`	Adopted	Risk annotation on a model.
`it6:Modality`	Adopted	Input modality.
`it6:trainedOn`, `it6:testedOn`	Adopted	Training and test dataset links.
Bridge constructs absent from MLDCAT-AP		
`daimo:AIAssetOffering`	Bridge extension	Governed catalogue offering with attached ODRL policy.
`daimo:ExecutionAuthorization`	Bridge extension	Execution authorisation grounded in an agreement.
`daimo:ModelDeployment`	Bridge extension	Deployment relation between model, infrastructure, and invocable service.
`daimo:IOContract`	Bridge extension	Input/output formats and authentication conditions.
`daimo:DerivedArtifact`	Bridge extension	Governable output of an authorised run.
`daimo:AuditEvidence`	Bridge extension	Integrity and audit evidence oriented to compliance.
`daimo:CrossParticipantProvenanceRecord`	Bridge extension	Provenance aggregation across participant contexts.
`daimo:SharedEvaluationContext`	Bridge extension	Reified comparability conditions for evaluations.
`daimo:ParticipantRole`	Bridge extension	Participant role in dataspace exchange.

(`odrl:Agreement`); DAIMO uses it intensively both for policy attached to offerings and for agreements that authorise concrete executions. PROV-O[9] provides the provenance ontology over activities, entities, agents, bundles, and roles, and is the natural layer for tracing individual executions and their aggregation across participants. Pandit and Esteves[23] show how ODRL combines with DPV for health-data scenarios; DAIMO transposes that pattern to the AI domain. These vocabularies are mature and widely adopted, but by construction they are domain-agnostic: none is specific to artificial intelligence or to dataspaces.

Dataspace standards. Where the previous vocabularies describe artefacts, dataspace standards describe exchange itself. The focus shifts from isolated artefacts to governed exchange between autonomous actors. Franklin et al.[24] framed *dataspaces* as a response to persistent source heterogeneity. Otto et al.[25] and Cappiello et al.[26] turn data sovereignty, contracts, and controlled interoperability into explicit design assumptions. The *Dataspace Protocol*[3] is the W3C-CG specification that defines catalogue, contract-negotiation, and transfer-process messages; Eclipse Dataspace Components[2] is its reference implementation.

It is important to distinguish *framework* from *vocabulary*: EDC is a Java execution framework, not an ontology. The semantics that EDC implements come from DSP (protocol), ODRL (policies), and DCAT (catalogue); only a handful of concepts are genuine EDC-specific extensions, and DAIMO explicitly references `edc:ParticipantContext` for its usefulness in anchoring aggregated provenance to an administrative context. This clarification matters because the dataspace literature frequently conflates the two layers. The evaluated DAIMO version preserves the DSP terms used during development; evolution of DSP 2025-1[6] is a tracking point for future informative-alignment versions. Work on federated ecosystems such as ConSolid[27] further shows that the Semantic Web is evolving towards multi-actor scenarios where governance and interoperability must be designed jointly.

Executable ontology engineering in SWJ

Beyond the domain substrate provided by the four families above, a fifth recent thread informs how DAIMO should be evaluated. The *Semantic Web Journal* has recently published a set of ontologies that share an explicit methodological commitment to executable validation and reproducibility. Kurteva et al. present RePlanIT[28], an ontology for digital product passports of ICT equipment, with SHACL and industry-expert usability evaluation. Palma et al. operationalise soil information at global scale in GloSIS[29] through a modular profile aligned with

SOSA/SSN and ISO 28258. Woods et al. connect ontology engineering with natural-language processing for industrial maintenance activities[35]. Bareedu et al. derive SHACL rules from OPC UA industry standards[36]. Ferranti et al. formalise Wikidata property constraints through SHACL and SPARQL[37]. Robaldo and Batsakis[38] analyse the interplay between SHACL validation and inference on the Time Ontology. Penteado et al.[39] study methodologies for publishing linked open government data. Together, these works establish the expectation—which DAIMO adopts as binding—that an ontology with operational ambitions should present reproducible evidence of correctness, not only an axiomatic description.

The comparison is now condensed into a reuse matrix. The table lets the reader see, in one view, which families cover publication, policy, invocation, traceability, and evaluation, and why DAIMO is formulated as an integration profile rather than as a substitute for those vocabularies.

A horizontal reading of Table 2 shows that no individual artefact simultaneously covers all five dimensions; a vertical reading confirms that each column is natively covered by at least one reusable vocabulary. DAIMO is therefore designed as an integration profile that ties those five capabilities together through verified axiomatic alignments, without duplicating what mature vocabularies already solve.

Methodology and requirements

DAIMO adopts the *Linked Open Terms* (LOT) methodology[40] for two reasons. First, its industrial orientation: LOT prescribes a lightweight flow with deliverables in each of its four phases (requirements specification, implementation, publication, and maintenance). Second, LOT treats reuse as a first-class phase rather than a by-product, which fits the reuse-first rule that the problem imposes. The subsections below describe the operational scenario that guided requirements elicitation (§), the competency questions and resulting non-functional requirements (§), and the three core design decisions that shaped the ontology (§).

Illustrative scenario

Requirements specification starts from an anonymised operational scenario that later acts as the case study (§). Provider A publishes a geospatial risk-prediction model in a federated catalogue managed by Platform C. Municipal Consumer B needs to invoke the model to support operational decisions after an extreme event, but cannot do so without first discovering it under a policy compatible with public use, verifying that it meets a minimum quality threshold under a shared evaluation context, negotiating an agreement that authorises its concrete executions, and invoking it through an authenticated endpoint. Scientific Evaluator D compares the model with a baseline under the same evaluation context. Compliance Actor E audits executions and archives the corresponding evidence. The names, models, endpoints, metrics, and dates in the scenario are synthetic by design: the case is not presented as a deployed pilot, but as a reproducible demonstrator of representational adequacy.

This scenario synthesises the five operational tasks that a catalogue-dataspace ontology must support simultaneously (publication, discovery, invocation, traceability, and evaluation) and involves five clearly differentiated actor types whose needs are complementary but not identical. It is plausible: it does not depend on hypothetical technology and can be instantiated with the reused vocabularies plus the extensions that DAIMO introduces.

Requirements specification

The requirements were elicited from three complementary sources: the scenario above, the MLDCAT-AP 3.0.0 specification and implementation notes, and the analysis of selected Eclipse EDC extension terms used in dataspace platforms. From them we formulated twenty-three competency questions, organised in five categories by actor and lifecycle stage. Category R (registration and publication, five questions) covers provider needs; category D (discovery and selection, four questions), consumer needs; category E (execution and auditability, five questions), operator and governance needs; category V (evaluation and reproducibility, five questions), evaluator needs; and category G (governance bridge, four questions), the multi-actor questions that require DAIMO-native semantics. The full set of twenty-three questions in natural language appears in Appendix .

The first synthesis links actors and question families. This table appears before technical traceability to fix who formulates each need and to prevent the CQs from reading as an abstract list detached from the scenario.

The second step traces each requirement family to the modelling commitment that satisfies it. This table justifies the main ontology blocks and connects the LOT specification to the evaluation artefacts used later.

The operational flow of federated execution is interpreted in DAIMO as an auditable semantic chain, not as a runtime architecture. Discovery is represented by governed offerings and CQs D1–D4; negotiation by `odrl:Agreement` and `ExecutionAuthorization`; deployment by `ModelDeployment`, `dcat:DataService`, and `IOContract`; inference by `it6:Run` and `DerivedArtifact`; and operational clearing by `AuditEvidence` and `CrossParticipantProvenanceRecord`. This interpretation preserves the sovereignty premise: DAIMO can describe executions on the data provider's node, on a neutral node, or on agreed infrastructure, but it does not prescribe whether the model moves to the data, the data moves to the model, or the execution takes place inside a specific isolation mechanism.

Term enumeration was treated as an intermediate artefact between competency questions and the TBox. Table 5 does not introduce new semantics: it normalises the operational vocabulary used in the scenario, records whether each term is reused or specialised, and prevents broad themes from becoming DAIMO-native classes without need. The full formal definitions remain in the ontology through `skos:definition`, `skos:example`, domains, ranges, and shapes.

The CQ set combines direct queries, subclass or subproperty paths, filters, aggregations, and negation-as-failure. This variety is deliberate: a profile that relied only on direct queries would not exercise the part of the artefact that makes OWL semantics valuable, and

Table 2. Reuse matrix against the required operational attributes. ● = covered natively; ○ = partially covered; — = not covered.

Family / artefact	Publication	Policy	Invocation	Traceability	Evaluation
Model cards, factsheets, datasheets [10–12]	○	—	—	—	○
ML-Schema, MEX, OntoDM, DMOP [14–17]	—	—	—	○	●
AIO, ITO, model-dataset management [18–20]	—	—	—	○	●
MLDCAT-AP 3.0.0 [4]	●	○	—	●	●
DCAT 2 [7]	●	—	○	—	—
ODRL 2.2 [8]	—	●	—	—	—
PROV-O [9]	—	—	—	●	○
DSP / EDC [2,3]	○	○	●	—	—
DAIMO (this work)	●	●	●	●	●

Table 3. Scenario actors and competency-question categories.

Actor	Category	Need	CQs
Model provider	R	Publish a model as a governed asset with identity, licence, policy, and invocation contract	CQ-R1–CQ-R5
Model consumer	D	Discover models by task, domain, policy, and quality threshold	CQ-D1–CQ-D4
Platform operator	E	Invoke services, trace executions, and reconstruct operational evidence	CQ-E1–CQ-E5
Evaluator	V	Compare results under a shared context and inspect reproducibility	CQ-V1–CQ-V5
Governance (multi-actor)	G	Bridge offer, deployment, authorisation, and cross-participant provenance	CQ-G1–CQ-G4

Table 4. Traceability between operational requirements, modelling commitments, and evidence.

Requirement family	Question the profile must answer	Primary DAIMO commitment	Evidence
Publication	Is this model a governed asset in a particular catalogue context?	Governed offering record linking model, provider, catalogue context, and applicable policy.	CQs R1–R5, G1
Discovery	Can a consumer filter models by task, domain, policy, and evaluation threshold?	MLDCAT-AP model/task/evaluation semantics plus policy links and evaluation context.	CQs D1–D4
Invocation	Which service, authentication method, and I/O contract apply to a selected model?	Deployment description plus service and I/O-contract commitments, allowing several endpoints per hosted model.	CQs E1, G2
Authorisation	Which agreement authorised a concrete execution by a concrete participant?	Negotiated agreement linked to the participant role and execution it covers.	CQs E2, G3; INV-2, INV-4
Auditability	Can a governance actor reconstruct artefacts, evidence, and provenance across contexts?	Derived artefact, evidence record, and cross-participant provenance bundle.	CQs E4, E5, G4; INV-1
Evaluation	Are measurements comparable under the same task, dataset version, protocol, and seed?	Shared evaluation context capturing task, dataset version, protocol, and seed.	CQs V1–V5

a profile that required materialised reasoning for every question would be impractical on real endpoints. The non-functional requirements complement the functional ones and fix: OWL 2 DL profile (for decidability and HermiT compatibility), CC-BY 4.0 licence for the ontology and Apache 2.0 for validation code, persistent identifiers under `https://w3id.org/pionera/daimo`, standard RDF serialisations, and accessible SHACL validation.

Core design decisions

Three structural decisions shape the rest of the design.

Decision 1: reuse preferentially. DAIMO reuses MLDCAT-AP classes directly for everything that MLDCAT-AP already models precisely. `it6:MachineLearningModel` is the model class; `it6:ComputerInfrastructure` describes the execution environment; `it6:Run`, `it6:Flow`, and `it6:Evaluation` capture respectively the concrete execution, the associated pipeline, and the resulting measurement; `dcat:Dataset`, `dcat:Distribution`, and `dcat:DataService` publish the asset; `odrl:Offer` and `odrl:Agreement` express policy and agreement. Introducing DAIMO-native equivalents for any of these classes would violate Gruber's principle

Table 5. Initial glossary of core domain terms and their ontological realisation.

Domain term	Operational meaning	Ontological realisation
AI model	Trained or packaged artefact that can be published, evaluated, and invoked.	Reused through the MLDCAT-AP model class; DAIMO does not duplicate the model class.
Governed offering	Act of registering a model as an available asset under policy in a federated catalogue.	`AIAssetOffering` links model, agent, and `odrl:Offer`; aligned with `dcat:CatalogRecord`.
Participant	Legal or organisational agent acting inside the dataspace.	`foaf:Agent` for the agent; `ParticipantRole` for the contextual role.
Participant role	Role played with respect to the asset: provider, consumer, operator, evaluator, or governance.	Five non-disjoint subclasses of `ParticipantRole`, because one agent may hold several roles.
Participant context	Administrative scope in which publication, negotiation, operation, or audit happens.	Reused as `edc:ParticipantContext`; DAIMO uses it to scope roles and provenance.
Policy and agreement	Usage conditions of an offer and result of an access negotiation.	Reuse of `odrl:Offer` and `odrl:Agreement`; DAIMO specialises only execution authorisation.
Model deployment	Hosted instance serving a model through one or more services.	`ModelDeployment` connects model, infrastructure, DCAT service, and I/O contract.
Service or endpoint	Accessible interface used to invoke a deployment.	Reused as `dcat:DataService`; DAIMO adds the link from the deployment.
I/O contract	Input format, output format, authentication, and optional schemas.	`IOContract` with `inputFormat`, `outputFormat`, `authMethod`, and optional schemas.
Authorised execution	Concrete invocation covered by a valid agreement.	`it6:Run` for the execution and `ExecutionAuthorization` for the agreement authorising it.
Derived artefact	Governable output produced by an execution.	`DerivedArtifact`, linked to the run and to the authorisation under which it was produced.
Audit evidence	Verifiable record supporting an executed activity.	`AuditEvidence` with activity, signer, timestamp, and SPDX checksum.
Cross-participant provenance	Provenance narrative spanning at least two participant contexts.	DAIMO cross-participant provenance class as a PROV bundle with explicit contexts.
Evaluation context	Conditions that make metrics from two models comparable.	`SharedEvaluationContext` fixes task, dataset, version, protocol, seed, and flow.
Evaluation and metric	Performance measurement used to compare models.	Reuse of `it6:Evaluation`; DAIMO links evaluation to the shared context.

of minimal ontological commitment and fracture the DCAT-AP ecosystem without gain; a DAIMO-native class is justified only when the union of reused vocabularies leaves a concrete gap that prevents answering at least one competency question.

This decision organises the conceptual model without turning each theme into a new class. The AI model remains `it6:MachineLearningModel`; data resources remain `dcat:Dataset` and `dcat:Distribution`; infrastructure remains `it6:ComputerInfrastructure`; metrics remain `it6:Evaluation` and `it6:EvaluationMeasure`; and governance is represented through ODRL policies, DAIMO authorisations, participant roles, and auditable evidence. Lineage is not reified as a monolithic class, but as a queryable chain connecting data, runs, deployments, derived artefacts, and PROV bundles. For the same reason, common candidates such as `AIModel`, `DataSet`, `Algorithm`, `ModelVersion`, `hasOwner`, `hasLicense`, frameworks, or hyperparameters are reused from MLDCAT-AP, DCAT, ODRL, PROV-O, or ML-Schema, or left to external profiles when no core competency question requires them. This keeps DAIMO decoupled from concrete implementation frameworks such as PyTorch or TensorFlow and avoids importing IDS or Gaia-X models as core dependencies; those ecosystems remain adoption contexts reachable through DCAT, ODRL, PROV-O, DSP, and the profile alignments.

Decision 2: separation into three complementary layers. The architecture distinguishes three layers that DAIMO connects but does not replace. Layer A is the dataspace plane: DSP with the specific EDC extensions that DAIMO references. Layer B is the platform-governance plane: a dataspace platform with its federated-registry mechanisms and operational policies. Layer C is the AI/ML asset plane: MLDCAT-AP and the vocabularies that support it. DAIMO is the integration profile that connects A, B, and C; it is not a fourth parallel layer that replicates functionality, but a bridge that makes the articulation between the three queryable.

Decision 3: one class per gap, not per theme. Given reuse preference, each class introduced by DAIMO must justify its existence by a concrete gap in the union of the reused vocabularies and by at least one competency question that cannot be answered without it. Strict application of this criterion produces nine top-level classes (`AIAssetOffering`, `ParticipantRole`, `ModelDeployment`, `IOContract`, `ExecutionAuthorization`, `DerivedArtifact`, `CrossParticipantProvenanceRecord`, `AuditEvidence`, and `SharedEvaluationContext`) plus five role subclasses. Any candidate class failing both conditions simultaneously is discarded.

The DAIMO ontology

Modular architecture and metrics

DAIMO models the operational lifecycle of an AI model asset inside a governed dataspace: publication as a catalogue record, policy- and quality-sensitive discovery, controlled invocation through typed services, traceability of individual executions, and contextualised, reproducible evaluation. The scope is deliberately restricted to this operational chain. Orthogonal responsibilities, such as detailed training-pipeline modelling, fine-grained privacy-policy semantics, or formal PROV derivation theory, are already covered by consolidated vocabularies whose duplication would fragment the ecosystem. DAIMO instead contributes the constructs needed to articulate those vocabularies coherently when an AI model circulates between autonomous participants.

The ontology is structured in three complementary modules, distributed in separate Turtle files so that a consumer can load only what it needs. The **core module** (`daimo-core.ttl`) contains the fourteen classes and twenty-nine object properties plus eight DAIMO-native datatype properties, the global disjointness axioms, and the property characterisations (functional, asymmetric, inverse) that capture TBox-level type invariants. The **alignment module** (`alignment.ttl`) encapsulates eight external `rdfs:subClassOf` declarations and five external `rdfs:subPropertyOf` declarations that connect DAIMO to reused vocabularies, plus minimal declarations of external terms referenced by DAIMO, each with an `rdfs:seeAlso` to the corresponding source specification; across the combined package there are six DAIMO subproperties to external vocabularies because `hasOffering` $\sqsubseteq$ `foaf:isPrimaryTopicOf` is declared in the core as the inverse-side accessor of `offersModel`. The **SHACL module** (`daimo-shapes.ttl`) provides the eighteen profile `sh:NodeShapes`: nine structural, three conformance shapes over reused classes, and six cross-class invariants through `sh:SPARQLConstraint`. The module is declared as an independent `owl:Ontology` with its own `owl:versionIRI`, so the validation layer can be released and referenced as an artefact with its own lifecycle.

The separation into three modules is not accidental. The core can be consumed by an application that needs only DAIMO primitives; the alignment module is loaded when a reasoner should infer relations between DAIMO and reused vocabularies; the SHACL module is loaded when a concrete graph should be validated against the profile.

The same modular design is summarised as an implementation profile to distinguish ontological decisions from operational conditions. The table connects language, IRIs, validation, and access with the declared scope, and avoids interpreting technical compatibility as evidence of production deployment.

Before turning to the metrics, Figure 1 summarises the organisation of the DAIMO package. It should be read from left to right: core, alignments, and SHACL are distributable modules of the artefact, while the vocabularies on the right are reused resources. DSP appears as an exchange protocol and EDC as an implementation referenced by the scenario, not as new functional layers.

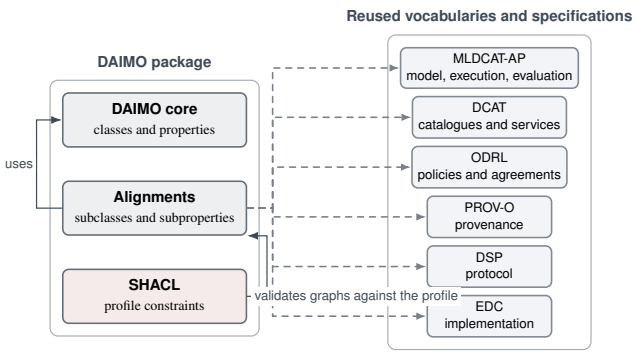


Figure 1. Modular organisation of the DAIMO package and reused resources.

Artefact metrics. Before reviewing the individual classes that populate the three modules of §, we report the size and expressivity of the artefact globally. Table 7 extracts the metrics directly from the Turtle files and from the reports produced by the validation suite (§). The numbers let an interested reader estimate the weight of the artefact before consuming it and compare it with ontologies previously published in the same family.

To make those counts auditable, an OntoMetrics-style breakdown is kept separate from the summary above. The distinction is useful because it avoids inflating the figures with external declarations loaded only for alignment and makes clear what is counted as a DAIMO-native term.

Classes and alignments

The nine top-level native classes, plus the five `ParticipantRole` subclasses, constitute the minimal set produced by the three core design decisions. Table 9 acts as an exhaustive inventory: class, alignment, role, and associated competency questions. The following figures do not duplicate that inventory; they provide a conceptual reading of the set and an axiomatic reading of the main alignments.

Figure 2 provides a functional reading of the complete ontology. It groups bridge classes and reused resources by their role in the publication, invocation, authorisation, evidence, and evaluation chain. This organisation helps read DAIMO as a relational profile rather than as a flat list of terms.

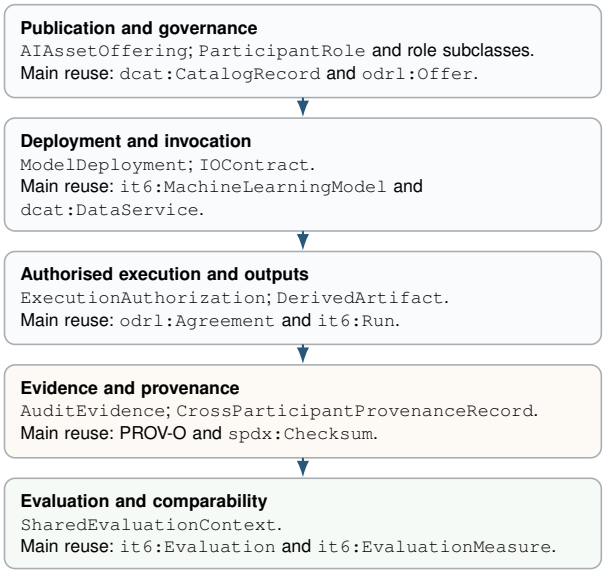


Figure 2. Global functional view of DAIMO by conceptual groupings.

Figure 3 complements the global view at the axiomatic level. The upper panel separates DAIMO bridge classes and reused classes connected by `rdfs:subClassOf`; the lower panel retains only the main operational relations that articulate the profile.

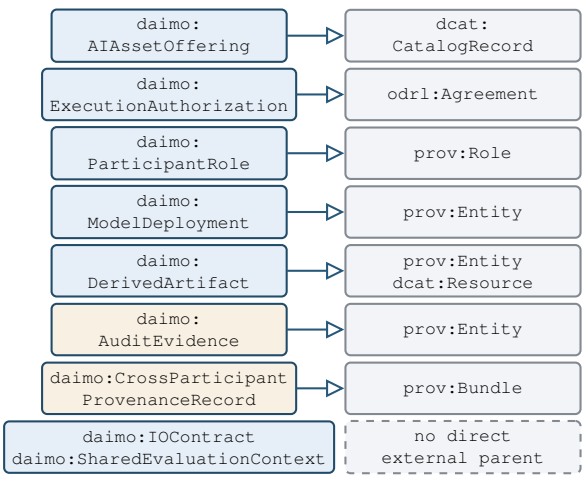
Table 6. DAIMO technical implementation profile.

Pillar	DAIMO implementation	Scope decision
Language, serialisation, and IRIs	OWL 2 DL core in Turtle; RDF/XML, JSON-LD, and N-Triples serialisations prepared for publication; canonical namespace `https://w3id.org/pionera/daimo`.	Local reproducibility does not depend on an active persistent endpoint.
Conceptual implementation views	Model metadata: MLDCAT-AP; data and catalogue: DCAT; execution and infrastructure: MLDCAT-AP plus `ModelDeployment`/`IOContract`; governance and trust: ODRL, authorisations, contextual evaluation, and evidence.	These are functional views, not new ontology modules such as `ModelMetadata` or `GovernanceTrust`.
Critical relations	`it6:trainedOn` is reused; formats and schemas are expressed through `IOContract`; deployment reaches endpoints through `exposedAs` and `dcat:DataService`; regulatory conformance can be expressed through `dct:conformsTo` or specific ODRL/SHACL profiles.	No duplicate properties such as `trainedOn`, `deployedIn`, or `compliesWith` are added when a standard or profileable path already exists.
Validation and reasoning	Structural SHACL, SHACL over reused classes, six SHACL-SPARQL invariants, HermiT, OWL-RL, OOPS!, negative tests, and 23 SPARQL queries.	No universal metric such as accuracy is required; mandatory metrics depend on the evaluation context and the CQ exercised.
Storage and access	The RDF package can be loaded into standard triplestores and exposed through SPARQL; the competency-query set demonstrates the access pattern.	The article does not claim a production GraphDB/Fuseki endpoint; it claims technical compatibility with such deployment.

Table 7. Main DAIMO artefact metrics.

Dimension	Value
Declared logical profile	OWL 2 DL
DAIMO top-level classes	9
DAIMO subclasses (`ParticipantRole`)	5
Total DAIMO-specific classes	14
Object properties (`owl:ObjectProperty`)	29
Datatype properties (`owl:DatatypeProperty`)	8
Functional properties (`owl:FunctionalProperty`)	27
Asymmetric properties (`owl:AsymmetricProperty`)	5
Explicit inverse pairs (`owl:inverseOf`)	5
`rdfs:subClassOf` to externals	8
`rdfs:subPropertyOf` to externals	6
Named disjointness axiom	1
Total `sh:NodeShape`s	18
completeness (one per DAIMO class)	9
conformance over reused classes	3
cross-class SPARQL invariants	6
Competency questions	23
OOPS! pitfalls (Critical/Important/Minor)	0 / 0 / 2

(a) External alignments



(b) Main operational relations

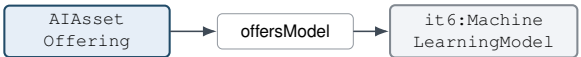


Table 8. OntoMetrics breakdown restricted to the DAIMO namespace.

Axiom / declaration type	Count
`owl:Class` declarations, DAIMO-native	14
`owl:ObjectProperty` declarations, DAIMO-native	29
`owl:DatatypeProperty` declarations, DAIMO-native	8
`rdfs:subClassOf`, DAIMO-native subjects	13
`rdfs:subPropertyOf`, DAIMO-native subjects	6
`rdfs:domain`, DAIMO-native properties	37
`rdfs:range`, DAIMO-native properties	37
`owl:FunctionalProperty`, DAIMO-native properties	27
`owl:AsymmetricProperty`	5
`owl:inverseOf` pairs	5
`owl:AllDisjointClasses` named axioms	1
`owl:equivalentClass`	0
`owl:disjointWith` binary axioms	0

The following paragraphs explain the modelling decisions by function. They do not repeat the full traceability of Table 9; they concentrate on why each grouping exists and which semantic boundary it preserves.

Publication and governance.
`AIAssetOffering` represents the act of registering a model as a governed offer in a dataspace catalogue. The class does not replace the model: the model remains `it6:MachineLearningModel`, reused from MLDCAT-AP, and the offering only contextualises its publication with provider, ODRL policy, and catalogue metadata. `ParticipantRole` captures the contextual role an agent plays with respect to the asset; its five subclasses (`ModelProvider`, `ModelConsumer`, `PlatformOperator`, `Evaluator`, and `GovernanceActor`) are not disjoint, because one agent may play several roles in different contexts. The profile is not a catalogue of catalogues: it models only the governed offering of AI models within the exchange described.

Table 9. Native classes, axiomatic alignments, and competency questions.

Class	Alignment	Role	CQs
daimo:AIAsset Offering	dcat:CatalogRecord	Catalogue record that publishes a model as a governed offer in a dataspace; carries an ODRL policy through daimo:hasOfferPolicy.	R1, R2, R5, G1
daimo:ParticipantRole (+5 non-disjoint subclasses)	prov:Role	Role a participant plays with respect to an AI asset within a dataspace context.	R5
daimo:Model Deployment	prov:Entity	Running or hosted instance of a model, exposed as a dcat:DataService over an it6:ComputerInfrastructure.	E1, E3, G2
daimo:IOContract	(no alignment)	Minimal actionable contract: I/O formats, authentication method, optional schemas.	R4, E1, G2
daimo:Execution Authorization	odrl:Agreement	ODRL agreement derived from a DSP negotiation that authorises concrete executions through daimo:authorizesRun.	E2, E5, G3
daimo:DerivedArtifact	prov:Entity, dcat:Resource	Governed, catalogueable output produced by an execution under authorisation.	E5, G4
daimo:CrossParticipant ProvenanceRecord	prov:Bundle	Aggregation of activities and entities across participant contexts forming an audit narrative.	G4
daimo:AuditEvidence	prov:Entity	Compliance-oriented evidence record with structured spdx:Checksum, signer, and timestamp.	E4
daimo:SharedEvaluation Context	(no alignment)	Reified comparability conditions: task, dataset, version, protocol, and random seed.	V1, V2, V3

Deployment and invocation.
ModelDeployment and IOContract connect the reused model to the invocation point. The deployment identifies the hosted or running instance of the model and may be exposed through one or several dcat:DataServices; the I/O contract describes formats, authentication, and optional schema references. DAIMO records where and under which syntactic conditions a model can be invoked, but it does not implement the runtime, enforcement engine, container orchestration, or TEE mechanisms. It also does not by itself resolve fine-grained semantic compatibility between input and output schemas; that verification belongs to dataspace-specific profiles or shapes.

Authorisation and derived artefacts.
ExecutionAuthorization reifies the agreement that authorises concrete executions, relying on ODRL for permissions and restrictions, and DerivedArtifact represents the governable output produced by an it6:Run. This separation avoids conflating three planes: negotiated policy, execution activity, and produced result. DAIMO describes the semantic chain that lets one ask under which authorisation an artefact was produced; it does not model the operational mechanism that decides or executes access.

Evidence and provenance.
AuditEvidence records verifiable evidence about an activity through integrity, signer, and timestamp; CrossParticipantProvenanceRecord aggregates activities and entities across participant contexts to form an audit narrative. PROV-O supplies the entity–activity–bundle pattern; SPDX supplies the structured checksum. DAIMO does not claim automatic compliance or external validation of that evidence: it provides the relations needed for evidence to be represented and queried.

Shared evaluation.
SharedEvaluationContext reifies the conditions that make two evaluations comparable: task, dataset, version, protocol, and seed. Measurements remain it6:Evaluation and it6:EvaluationMeasure; DAIMO adds only the context that prevents interpreting a metric as an absolute property of the model. This decision keeps a universal theory of bias, fairness, explainability, or security outside the core: those dimensions can be expressed as contextualised measures or specialised profiles when a use case requires them.

Compact alignment defence.
External alignments follow the same minimal-duplication criterion. AIAssetOffering aligns with dcat:CatalogRecord, not dcat:Dataset, because the offering is the contextual record that describes a model in a catalogue, not the model or a dataset; offersModel maintains the link to it6:MachineLearningModel. ExecutionAuthorization aligns with odrl:Agreement because the authorisation is the effective agreement that covers executions, not a derived artefact of another agreement. ParticipantRole aligns with prov:Role with caution: DAIMO adds administrative scope through participant context, but preserves the reading of role as a contextual qualification of participation.

The remaining alignments preserve the PROV-O entity–activity distinction. ModelDeployment is a prov:Entity, not an activity; DerivedArtifact is both prov:Entity and dcat:Resource, because it is produced by a run and can be catalogued; CrossParticipantProvenanceRecord is a prov:Bundle, not a generic collection; and AuditEvidence is a prov:Entity, not the act of auditing. IOContract and SharedEvaluationContext have no direct external parent by design: the former is neither a service nor a standard, and the latter is not a task or an evaluation, but the reification of comparability conditions.

Axiomatic foundations

A taxonomy alone does not fix meaning; it only names the types. DAIMO therefore combines the class hierarchy with an axiomatic infrastructure that fixes what each class is, what it is not, and how conforming instances must behave. Three complementary strata make up this infrastructure: OWL axioms over classes and properties, SHACL nodal constraints over instances, and SHACL-SPARQL cross-class

invariants. The OWL stratum is described here; the SHACL strata are reported in Section as part of validation.

On the OWL stratum, disjointness between the nine DAIMO top-level kinds is declared through the named axiom `daimo:TopLevelKindsDisjointness`, an instance of `owl:AllDisjointClasses`. The five `ParticipantRole` subclasses are deliberately excluded: one legal agent may hold several roles at once (provider and operator, for instance), which reflects that roles are context-dependent rather than mutually exclusive.

The disjointness core is complemented by property characterisations. Twenty-seven properties are declared functional where the corresponding structural shape also imposes maximum cardinality, so cardinality is represented in both the logical and validation layers. Five lifecycle relations are asymmetric, which formally captures that a deployment is not the model it deploys, an authorisation does not coincide with the execution it authorises, and so on. Five inverse pairs support bidirectional traversal without forcing the publisher of the graph to materialise both directions.

The SHACL stratum splits into two families. Each DAIMO governance class carries a structural nodal shape that prescribes which properties are mandatory in a well-formed instance; these nine shapes are the syntactic guarantee that a publisher has supplied everything the semantic invariants need. Three conformance shapes (`OfferInDAIMOShape`, `MachineLearningModelInDAIMOShape`, and `RunInDAIMOShape`) establish which properties of reused classes are mandatory when a graph declares adherence to the DAIMO profile. The distinction between the two families is intentional: DAIMO does not override the underlying ontology, because that would fracture the ecosystem. Instead, it declares a commitment to a concrete subset of properties on the external classes, making the adherence criterion explicit without modifying the original definitions.

Above the shapes sit the six SHACL-SPARQL cross-class invariants (INV-1 through INV-6), examined in detail during validation. They encode rules whose meaning cannot be expressed as a local constraint on a single shape because the rules span several classes simultaneously. Listing 1 shows how the OWL axioms are encoded for the concrete case of `ExecutionAuthorization`: the subclass of `odrl:Agreement`, the asymmetry of `authorizesRun`, the functionality of its inverse, and membership in the named disjointness axiom.

```
daimo:ExecutionAuthorization a owl:Class ;
   rdfs:subClassOf odrl:Agreement ;
   skos:definition "An ODRL agreement
       produced by a DSP contract
      negotiation that binds specific
          permissions to run invocations..."
          @en .

daimo:authorizesRun a owl:ObjectProperty ,
    owl:AsymmetricProperty ;
   rdfs:domain daimo:ExecutionAuthorization
       ;
   rdfs:range it6:Run .

daimo:authorizedBy a owl:ObjectProperty ,
    owl:FunctionalProperty ;
   owl:inverseOf daimo:authorizesRun ;
   rdfs:domain it6:Run ;
   rdfs:range daimo:ExecutionAuthorization .

daimo:TopLevelKindsDisjointness a owl:
    AllDisjointClasses ;
   owl:members ( daimo:AIAssetOffering
              daimo:ExecutionAuthorization
              daimo:ModelDeployment
              daimo:DerivedArtifact
              daimo:IOContract
              daimo:AuditEvidence
              daimo:
                  CrossParticipantProvenanceRecord

              daimo:SharedEvaluationContext
              daimo:ParticipantRole ) .
```

Listing 1: Core axioms around `ExecutionAuthorization`.

Reuse surface. Table 10 summarises the reused vocabularies and their concrete use. Three observations matter. First, the alignments to DCAT and ODRL target standard vocabularies rather than implementation-internal entities. Second, DAIMO maintains informative mappings to DSP through `skos:closeMatch` and `skos:related`, not subsumption: DSP is a messaging protocol rather than an asset ontology, and subsuming DAIMO classes under DSP constructs would introduce spurious typings. Third, of the EDC extension only `edc:ParticipantContext` appears explicitly, where it anchors aggregated provenance to an administrative context.

Three subproperties that might seem natural (`authorizesRun` $\sqsubseteq$ `prov:used`; `grantedTo` $\sqsubseteq$ `prov:qualifiedAssociation`; `evidenceOf` $\sqsubseteq$ `prov:hadActivity`) are deliberately not declared, because each would produce unintended typings on authorisations, agents, or evidence under RDFS closure. The entailment-verification check reported below catches errors of this kind.

Evaluation

Evaluation distinguishes two objects: DAIMO as an ontology artefact and the comparative evaluation of models that DAIMO can describe. The former is examined technically through formal consistency, OWL-RL closure, SHACL conformance, absence of critical pitfalls, alignment behaviour, and competency-question answering. The latter appears only inside the demonstrator as represented content under a shared context; it does not constitute an empirical benchmark of real models. The reported evidence is therefore executable over the artefact and over a controlled graph, not external expert validation or production deployment.

In LOT terms, fit-for-purpose adequacy is addressed in a bounded way through a realistic instance graph, executable SPARQL queries, and negative tests. Following the usual distinction between requirements coverage, formal correctness, constraint-based validation, and applied usefulness[48], we ask whether the profile represents the target exchange, whether the OWL artefact is formally coherent, whether SHACL constraints discriminate valid

Table 10. Vocabularies reused by DAIMO and the concrete use of each.

Vocabulary	Reused terms	Use in DAIMO
MLDCAT-AP 3.0.0 (`it6:`)	`MachineLearningModel`, `Task`, `Run`, `Flow`, `Evaluation`, `EvaluationMeasure`, `Benchmark`, `ComputerInfrastructure`, `Hardware`, `Library`, `HarmRisk`, `Modality`, `trainedOn`, `testedOn`	AI-asset plane; DAIMO does not redefine the model.
DCAT 2 (`dcat:`)	`Catalog`, `Dataset`, `Distribution`, `DataService`, `CatalogRecord`, `endpointURL`	Publication and discovery.
ODRL 2.2 (`odrl:`)	`Offer`, `Agreement`, `Permission`, `Prohibition`, `hasPolicy`, `target`, `assigner`, `assignee`	Policy, offer, and agreement.
PROV-O (`prov:`)	`Activity`, `Entity`, `Agent`, `Bundle`, `Role`, `wasGeneratedBy`, `wasAssociatedWith`	Provenance and roles.
FOAF (`foaf:`)[44]	`primaryTopic`	Subproperty target for linking an offering record to the model it is about.
SPDX (`spdx:`)[45]	`Checksum`, `algorithm`, `checksumValue`	Audit-evidence integrity.
SKOS (`skos:`)[46]	`definition`, `example`, `closeMatch`, `related`	Term documentation and non-subsumptive mappings.
DSP (`dspace:`)	`ContractOffer`, `ContractNegotiation`, `TransferProcess`	Informative `skos:closeMatch`/`skos:related` mappings; not subsumption.
EDC (`edc:`)	`ParticipantContext`	Only EDC-specific extension referenced.

from invalid graphs, and whether the competency questions are executable over the materialised graph. The analysis follows the five-dimension pattern of RePlanIT[28]: scenario instantiation (§), formal verification through reasoning, entailment checking, and OOPS! scanning (§), SHACL instance validation (§), execution of the twenty-three competency questions (§), and verification of reuse alignments (§). All five dimensions are reproducible with the evaluation package that accompanies the artefact.

Illustrative demonstrator

The scenario introduced in § is instantiated on DAIMO as a controlled demonstrator. The resulting graph contains 225 data triples that exercise every native class at least once and over which the twenty-three competency queries reported in § are executed. All organisations, model names, dataset versions, metrics, and endpoints are anonymised or synthetic. The demonstrator is not a deployment study, a gold standard, or a validation against real model catalogues; its function is to examine representational adequacy and make the validation artefacts executable.

Provider A publishes three governed model offerings in a federated catalogue: two versions of the same geospatial risk-prediction model and one read-only variant with a stricter policy. The offerings share task and domain descriptions but differ in the permissions attached to them. This part of the scenario exercises the distinction between a model as an AI asset and an offering as a contextual catalogue record with its own usage conditions.

The selected model is deployed on a described compute environment and exposed through two service interfaces: a REST endpoint with JSON input and bearer-token authentication, and a gRPC endpoint with protobuf and mutual TLS. The multi-endpoint usage exercises the modelling decision that a single deployment may expose several services and contracts, rather than forcing one endpoint per deployment.

Consumer B obtains an execution authorisation with explicit validity dates, invokes the model, and receives a derived prediction artefact. The execution is linked to audit evidence containing an integrity record, signer, and timestamp, so that a governance actor can reconstruct how the artefact was produced and under which authorisation. A cross-participant provenance record aggregates the chain across provider, consumer, and platform contexts. Evaluator D publishes two synthetic evaluation results under the same benchmark, protocol, and random seed, allowing the two model versions to be compared under shared conditions.

Figure 4 summarises the demonstrator as a phase flow. The phases are read from top to bottom, from publication to shared evaluation, and each retains only the constructs needed to explain DAIMO's representational role. Fine-grained property detail remains in the text and in the query traceability.

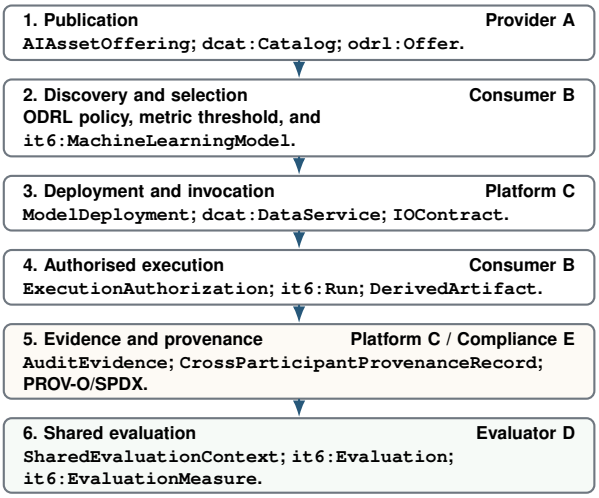


Figure 4. Phase-based view of the bounded multi-participant demonstrator.

Formal verification

HermiT, invoked through `owlready2`, confirms that the union of the core, the alignments, and the example graph is logically consistent with no unsatisfiable classes; reasoning takes 1.25 s. OWL-RL materialisation (the `owlrl` library) derives 1 170 additional triples on top of 818 initial ones (1 988 in total) in 0.58 s, with no individuals typed as `owl:Nothing` and no unsatisfiable subclasses.

Logical consistency is necessary but not sufficient. An ontology may be consistent and still infer, for instance, that audit evidence is a provenance activity rather than an entity produced by an activity. We therefore added an entailment-verification check over the OWL-RL closure: for each DAIMO class, the check enumerates every inferred superclass and tests it against a list of forbidden ones. The report covers the fourteen DAIMO classes (nine top-level plus five role subclasses) and raises no warning. This check complements consistency reasoning by detecting modelling-inappropriate alignments that remain logically satisfiable.

The OOPS! scanner[50], queried through its REST API on the union of the core and alignments, reports zero Critical pitfalls, zero Important ones, and two Minor ones: P13 ("inverse relationships not declared") on 34 elements with no semantically meaningful inverse, and P04 ("isolated ontology elements") on seven locally declared external classes. Both arise from the reuse-by-default discipline the profile applies.

Structural-quality metrics are treated as diagnostic signals, not as absolute acceptance criteria. Counts such as hierarchy depth, taxonomic breadth, connectivity, explicit equivalences, or isolated elements help identify areas that deserve inspection, but they do not by themselves prescribe a modelling change. In OWL, a `rdfs:subClassOf` cycle may express implicit equivalence rather than an error; an `owl:equivalentClass` declaration may be a legitimate alignment; and disjointness should be introduced only when the ontological commitment is true in every admissible model of the domain. DAIMO therefore uses OOPS!, HermiT, OWL-RL, SHACL, negative tests, and competency questions as complementary layers: each observes a different risk, and none replaces modelling judgement.

SHACL validation: structural, invariants, and negative test bench

The SHACL layer contains three *shape* families, each answering a different validation question. Structural shapes ask whether every DAIMO-specific instance carries the mandatory properties of its class. Conformance shapes ask whether reused external classes satisfy the additional obligations that the profile imposes on them. SPARQL invariants ask whether relations among offering, policy, deployment, authorisation, run, derived artefact, and evidence remain coherent. Together, the three families move validation from local completeness to graph-level consistency: from "is this instance well-formed?" to "is this graph internally coherent?". The SHACL layer therefore acts as the profile's data contract; it does not introduce a parallel ontology or require universal lineage, model-format, or metric constraints that are not justified by the competency questions.

At implementation level, constraints are encoded as `sh:NodeShapes` targeting DAIMO classes or reused classes under the profile, and as embedded `sh:PropertyShapes` that fix cardinalities, datatypes, expected classes, patterns, or closed enumerations. In an operational flow, a provider may publish RDF or JSON-LD metadata for a model, a connector or catalogue service may execute the `daimo-shapes.ttl` layer, and the resulting `sh:ValidationReport` would indicate whether the graph conforms or which specific property violates the profile. This describes the expected consumption mode of the shapes; it is not presented as an already deployed production connector or as validation of the ontology TBox itself.

The nine structural shapes impose minimum completeness per class; the positive graph of 225 triples satisfies `conforms=True`. The three conformance shapes add operational obligations over reused classes: `OfferInDAIMOShape` requires every `odrl:Offer` used in DAIMO to declare `odrl:assigner` and `odrl:target`; `MachineLearningModelInDAIMOShape` requires policy, title, and identifier on `it6:MachineLearningModel`; and `RunInDAIMOShape` requires flow, algorithm, associated agent, and timestamp on `it6:Run`. Additionally, `sh:in` restricts `daimo:authMethod` to a closed vocabulary of nine tokens, and `sh:pattern` restricts `daimo:protocol` to recognised evaluation-protocol names.

Above this structural stratum sit the six SHACL-SPARQL cross-class invariants. INV-1 guarantees consistency of the derivation-authorisation chain: every derived artefact must point to an authorisation that covers precisely the execution from which the artefact was derived. INV-2 requires the actor recorded in provenance for an execution to match the beneficiary in at least one covering authorisation. INV-3 checks data-plane coherence: a service exposed by a deployment must serve the same model the deployment declares, ruling out an endpoint that advertises one model and serves another. INV-4 ensures that an authorisation cannot justify an execution that started after the authorisation lapsed. INV-5 requires the policy attached to an offering to

govern the model named in that offering. INV-6 closes the same triangle on the provider side by checking consistency between the catalogue-record attribution and the policy assigner. The positive graph conforms to all six invariants without warnings.

Listing 2 shows the concrete encoding of INV-5 as `sh:SPARQLConstraint`, illustrating the `FILTER NOT EXISTS` pattern that captures the disjunction between policy-level and per-permission targets.

```
daimo:OfferingPolicyTargetInvariant a sh:
    NodeShape ;
   sh:targetClass daimo:AIAssetOffering ;
   sh:sparql [
      a sh:SPARQLConstraint ;
      sh:message "INV-5 violation: offering'
          s offersModel does not
               appear as odrl:target of the
                  attached policy."@en ;
      sh:prefixes daimo:_invariantPrefixes ;
      sh:select """
         SELECT $this ?model ?policy WHERE {
            $this daimo:offersModel ?model ;
                 daimo:hasOfferPolicy ?policy
                    .
            FILTER NOT EXISTS { ?policy odrl
                :target ?model }
            FILTER NOT EXISTS { ?policy odrl
                :permission/odrl:target ?
                model }
         }
      """ ;
   ] .
```

Listing 2: INV-5 encoded as a `sh:SPARQLConstraint`.

Shapes that conform on the positive graph are necessary but not sufficient to show that constraints are semantically discriminating. For this reason DAIMO also includes a negative graph of 118 triples encoding six deliberately invalid cases—`bad:INV1-artifact`, `bad:INV2-run`, `bad:INV3-deployment`, `bad:INV4-auth`, `bad:INV5-offering`, and `bad:INV6-offering`—each focused on one invariant. The test loads the ontology, shapes, and negative graph, runs SHACL, and verifies that validation fails globally and that each expected focus node appears in the report with the specific invariant message. All six invariants fire correctly on their corresponding violation.

Competency questions

The twenty-three competency questions are expressed as SPARQL queries and run over the OWL-RL materialised closure of the positive graph. The table below summarises coverage by category and preserves the observed row counts; row-level traceability appears in Table 15 in Appendix . All 23/23 queries return at least one row, verifying that the demonstration graph contains the information needed to answer every question with the primitives DAIMO declares; it does not evaluate the empirical quality of the models represented.

Two results deserve emphasis. CQ-R2 and CQ-E1 depend on the `subPropertyOf` relation between `daimo:offersModel` and `foaf:primaryTopic` and return no rows on a plain SPARQL engine without reasoning; CQ-R5 explicitly traverses the role hierarchy. The dependency is documented, not a defect: aligning `offersModel` under `foaf:primaryTopic` makes offerings discoverable through the standard DCAT descriptive property, a benefit that materialises only under inference. A consumer who needs to run those queries on a plain endpoint can materialise the closure beforehand or rewrite the queries to enumerate both properties. The second result is that CQ-G2 returns two rows—one per endpoint of the multi-protocol REST and gRPC deployment—confirming that modelling several services per deployment is representable and directly queryable from SPARQL.

Two fragments illustrate the formalisation pattern. Listing 3 shows CQ-G3: the query retrieves the authorisation that covers a concrete run, the authorised grantee, and the expiry time. The alignment $\texttt{ExecutionAuthorization} \sqsubseteq \texttt{odrl:Agreement}$ makes the resource interpretable as an ODRL agreement without duplicating contract properties in the query. Listing 4 shows CQ-V3: two evaluations are compared only because they share the same `SharedEvaluationContext`; ordering by metric value is meaningful inside that context rather than as a global ranking independent of dataset, protocol, or seed.

```
SELECT ?auth ?grantee ?expires WHERE {
  ?auth a daimo:ExecutionAuthorization ;
       daimo:authorizesRun
         <https://example.org/daimo-scenario/
             run-2026-04-20-legs> ;
       daimo:grantedTo ?grantee ;
       daimo:expiresAt ?expires .
}
```

Listing 3: CQ-G3: authorisation and agreement for a specific run.

```
SELECT ?model ?value WHERE {
  ?eval a it6:Evaluation ;
       it6:evaluates ?model ;
       daimo:usesEvaluationContext
         <https://example.org/daimo-scenario/
             evalctx-climatebench-v1-2026-1-
             holdout> ;
       it6:hasEvaluationMeasure ?m .
  ?m it6:hasValue ?value .
}
ORDER BY DESC(?value)
```

Listing 4: CQ-V3: ranking of models under a shared evaluation context.

Verified reuse. Reuse is verified directly on the alignment module. Eight `rdfs:subClassOf` and five `rdfs:subPropertyOf` declarations, listed in Table 9, connect DAIMO to its five parent vocabularies (DCAT, MLDCAT-AP, ODRL, PROV-O, and FOAF); class alignments are visualised in Figure 3. The sixth DAIMO subproperty towards an external vocabulary, $\texttt{hasOffering} \sqsubseteq \texttt{foaf:isPrimaryTopicOf}$, is declared in the core because it implements the inverse navigation of `offersModel`. Verification operates in two dimensions.

Table 11. Coverage of the 23 competency questions over the positive graph closed with OWL-RL.

Category	Queries (rows)	N	Relevant semantic operation or SPARQL
Registration and publication (R)	R1(3) R2(1) R3(1) R4(2) R5(2)	5	**R2** (`subPropertyOf` chain), **R5** (`subClassOf` path)
Discovery and selection (D)	D1(3) D2(2) D3(4) D4(1)	4	D2 uses negation-as-failure; D4 applies a numeric filter. D1 and D3 are direct queries.
Execution and auditability (E)	E1(4) E2(1) E3(2) E4(1) E5(1)	5	**E1** (`offersModel` $\sqsubseteq$ `foaf:primaryTopic`); E2 uses an agreement–run union.
Evaluation and reproducibility (V)	V1(1) V2(1) V3(2) V4(1) V5(4)	5	V2 uses `MAX` aggregation; V3 uses ordering.
Governance bridge (G)	G1(1) G2(2) G3(1) G4(1)	4	G3 exercises the DAIMO–ODRL bridge; G4 uses `GROUP_CONCAT`.

The first is *structural*: every native class that could have been defined in isolation is placed under an external parent whenever such a parent exists, and every new property extends a reused property whenever the latter already carries the intended meaning. No class is introduced for stylistic reasons, and `IOContract` and `SharedEvaluationContext` remain without an external parent only because no parent with truthful semantics exists (§).

The second is *semantic*: alignments must not produce unintended inferences. The entailment verification of § inspects the OWL-RL closure of each alignment and confirms that none produces a semantically incorrect superclass on any native class. The reasoner reports zero warnings over the fourteen classes. Three subproperty alignments towards PROV-O were deliberately excluded from the module for the same reason: each would have produced a forbidden typing under RDFS closure (§).

Non-functional requirement coverage

The non-functional requirements are evaluated over the ontology artefact and its validation workflow, not over a federated execution platform. In particular, scalability means growth in the number of governed exchange instances in the ABox while the DAIMO core remains fixed at fourteen native classes. The result therefore does not measure connector concurrency, online inference latency, or production throughput. It measures reproducible tractability for the operations the profile itself requires: RDF parsing, OWL-RL materialisation, SHACL validation, and SPARQL querying.

Non-functional coverage is summarised before the benchmark to separate the kind of evidence available from the risks that remain outside the artefact. Table 12 maintains this distinction between reproducible validation, potential interoperability, and the absence of production claims.

The benchmark is presented as bounded tractability evidence, not as operational performance proof. Table 13 preserves the reported times and counts for 100 and 1,000 synthetic units and shows that SHACL dominates validation cost.

Discussion

DAIMO provides an integration profile for representing AI models as governed assets in dataspaces. Its contribution is semantic and governance-oriented: it connects catalogue, policy, deployment, execution, evidence, and evaluation without standardising weights, internal architectures, or inference mechanisms. This position aligns DAIMO with recent *Semantic Web Journal* ontologies through reuse, executable implementation, and competency questions, but applies that pattern to the bridge between the AI catalogue and the exchange plane.

The relation to reused vocabularies is deliberately complementary. MLDCAT-AP remains the source of the model through `it6:MachineLearningModel`; DCAT provides catalogues and services; ODRL expresses offers and agreements; PROV-O structures provenance; DSP describes the negotiation protocol; and EDC is treated as a possible implementation, not as a domain ontology. DAIMO adds bridge relations only where that combination cannot query the full governed chain, for example offering, deployment, execution authorisation, and provenance across administrative contexts.

Semantic limitations delimit the profile's scope. DAIMO is not a catalogue of catalogues or a federated query engine. `IOContract` captures format, authentication, and optional schema references, but does not resolve fine-grained semantic compatibility between inputs and outputs. Likewise, DAIMO describes authorised and auditable executions, but does not implement runtime, federated enforcement, container orchestration, or TEE mechanisms. Those decisions belong to platforms or specialised profiles.

Several domain extensions also remain outside the core. `SharedEvaluationContext` allows comparison under the same task, dataset, version, protocol, and seed, but does not define a universal theory of bias, fairness, explainability, or security. Restrictions inherited from training datasets towards models or derived artefacts are not axiomatised as a general rule: they depend on contract, licence, jurisdiction, and training technique. When needed, they should be expressed through dataspace-specific SHACL or ODRL profiles.

Some integration commitments produce effects that consumers should know. The property `daimo:records` may infer MLDCAT-AP evaluations as PROV activities when they are grouped inside a `CrossParticipantProvenanceRecord`; this effect is useful for audit, but should be documented in integrations that combine DAIMO, MLDCAT-AP, and PROV-O. DAIMO is also not anchored to an upper ontology, because it is presented as an integration profile rather than as a reference ontology.

Validation limitations are equally important. The reported evidence is executable validation of the artefact and demonstrator graph: consistency, OWL-RL closure, SHACL, negative tests, OOPS!, competency questions, and bounded

Table 12. Non-functional requirement coverage and residual risk.

Requirement	Reproducible evidence	Status and residual risk
Logical consistency	HermiT reports consistency and zero unsatisfiable classes; OWL-RL materialises 1,170 triples without `owl:Nothing`; entailment verification inspects fourteen native classes with zero warnings.	Strong evidence at artefact level. Consistency is complemented with SHACL negative tests so constraints are not merely trivially satisfiable.
Interoperability	The profile uses RDF/OWL 2 DL, SHACL, and SPARQL; it reuses DCAT, MLDCAT-AP, ODRL, PROV-O, FOAF, SKOS, and SPDX through external IRIs and axiomatic alignments.	Strong evidence at artefact level. Operational interoperability will depend on effective publication and consumption in third-party infrastructures.
Scalability	The benchmark generates conforming synthetic graphs of 100 and 1,000 exchange units; both are SHACL conformant and answer the control queries.	Measured bounded coverage. SHACL is the dominant cost; no production throughput or 10,000-unit result is claimed without executing it.
Extensibility	The core remains minimal and SemVer-versioned; extensions are expected as complementary profiles, for example `daimo-discovery`, without absorbing ranking or runtime-execution logic.	Strong evidence at artefact level for controlled evolution. New concepts must justify that no reused vocabulary already provides them.
Modularity	Core, alignments, shapes, examples, queries, negative tests, and reports are separate artefacts; each module can be executed or reviewed independently.	Strong evidence at artefact level. Modular separation reduces the risk of breaking the core when external profiles are added.
Documentation	Each term includes SKOS labels and definitions; supplementary documentation traces classes, properties, CQs, invariants, and reproduction commands.	Strong evidence at artefact level for local review and reproduction. Public documentation quality should remain synchronised with future profile versions.

Table 13. Bounded synthetic scalability benchmark.

Units	Data triples	Closure triples	Parse (s)	OWL-RL (s)	SHACL (s)	SPARQL (s)	Conforms
100	8,053	16,248	0.468	5.478	10.374	0.080	Yes
1,000	80,053	147,648	4.660	53.672	135.010	0.356	Yes

synthetic scalability. It does not replace external expert review, does not prove production deployment, and does not constitute automatic AI Act compliance. Reported numerical results also depend on documented reasoning conditions, in particular RDFS inference during SHACL and OWL-RL closure before competency queries.

The practical implication is therefore a subsequent validation agenda: instantiate DAIMO over real MLDCAT-AP catalogues, test its consumption with dataspace connectors, review the profile with domain experts, and contrast high-risk cases against concrete regulatory requirements. None of these tasks is claimed as completed in this article; all are necessary steps to move from executable artefact validation to external validation and real deployment.

Future integration should preserve this boundary. Discovery, ranking, recommendation, federated execution, or user-interface applications may consume DAIMO graphs, but should not be absorbed into the core. The detail of complementary families and extension paths is kept in Appendix so the closing discussion remains concise.

Availability and sustainability

DAIMO is accompanied by the artefacts needed to inspect and reproduce the evaluation: OWL modules in Turtle, alignments, SHACL shapes, the demonstrator graph, competency queries, a negative test bench, and generated reports. The ontology and its documentation are released under CC-BY 4.0, while validation code is released under Apache 2.0. Reproducibility does not depend on a production endpoint.

Sustainability follows LOT Phase 4 through versioned IRIs, Dublin Core metadata, SemVer policy, and separation between core, alignments, and the SHACL layer. These elements support artefact availability and reuse; they are not presented as evidence of adoption, real deployment, or external validation.

Conclusions

DAIMO proposes a reuse-first contribution: it reuses MLDCAT-AP, DCAT, ODRL, PROV-O, DSP/EDC, FOAF, SKOS, and SPDX, and adds only bridge classes to articulate catalogue, policy, deployment, execution, evidence, and evaluation in dataspaces. The AI model is not redefined; it remains `it6:MachineLearningModel`.

The presented evidence is reproducible and bounded: OWL consistency, OWL-RL closure, SHACL conformance, negative tests, an OOPS! scan, twenty-three competency questions, and a bounded synthetic scalability benchmark. This evidence supports the coherence of the artefact and of the controlled demonstrator, but it does not amount to external validation, institutional adoption, or production deployment.

Future work should concentrate on validating DAIMO outside the controlled environment: instantiation against real MLDCAT-AP catalogues, qualitative review with experts, pilot integration with dataspace connectors, refinement of `IOContract`, finer separation between role type and role assignment, extension of `SharedEvaluationContext` to heterogeneous benchmarking protocols, and complementary profiles for reproducibility, monitoring, security, and compliance.

References

1. European Union (2024) *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)*. Official Journal of the European Union.
2. Eclipse Foundation. *Eclipse Dataspace Components (EDC)*, control-plane documentation.
3. International Data Spaces Association (2024) *Dataspace Protocol (DSP)*. W3C Community Draft.
4. SEMIC (2025) *MLDCAT-AP 3.0.0: Machine Learning Data Catalogue Application Profile*. European Commission.
5. SEMIC (2026) *MLDCAT-AP 3.1.0: Machine Learning Data Catalogue Application Profile*. European Commission, Candidate Recommendation, 13 May 2026.
6. Eclipse Dataspace Protocol (2025) *Dataspace Protocol Release 2025-1*. Eclipse Dataspace Working Group.
7. Albertoni, A. et al. (2020) *Data Catalog Vocabulary (DCAT) — Version 2*. W3C Recommendation.
8. Iannella, R., Guth, S. and Steyskal, R. (eds.) (2018) *ODRL Information Model 2.2*. W3C Recommendation.
9. Lebo, P., Sahoo, S. and McGuinness, D. (eds.) (2013) *PROV-O: The PROV Ontology*. W3C Recommendation.
10. Mitchell, M. et al. (2019) Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*.
11. Arnold, M. et al. (2019) FactSheets: Increasing Trust in AI Services through Supplier's Declarations of Conformity. *IBM Research Report*.
12. Gebru, T. et al. (2021) Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.
13. Soldatova, L.N. and King, R.D. (2006) An ontology of scientific experiments. *Journal of the Royal Society Interface*, 3(11).
14. Correa Publio, G. et al. (2018) ML-Schema: Exposing the semantics of machine learning with schemas and ontologies.
15. Keet, C.M., Fernández, J.D. and Panov, P. (2016) MEX vocabulary: A lightweight interchange format for machine learning experiments. *ISWC*.
16. Panov, P., Dzeroski, S. and Soldatova, L.N. (2008) OntoDM: An ontology of data mining. *ICDMW*.
17. Keet, C.M. et al. (2015) The data mining optimization ontology. *Web Semantics*, 32.
18. Joachimiak, M.P. et al. (2024) The Artificial Intelligence Ontology: LLM-assisted construction of AI concept hierarchies. *Applied Ontology*.
19. Blagec, K., Barbosa-Silva, A., Ott, S. and Samwald, M. (2022) A curated, ontology-based, large-scale knowledge graph of artificial intelligence tasks and benchmarks. *Scientific Data*, 9:322.
20. Novacek, J., Ahari, A., Müller, T., Reiter, S., Viehl, A. and Bringmann, O. (2024) Ontology-Supported AI Model and Dataset Management. *IEEE 22nd International Conference on Industrial Informatics (INDIN)*, 1–6.
21. Souza, R. et al. (2019) Provenance data in the machine learning lifecycle in computational science and engineering. *WORKS*.
22. Schmidt, M. et al. (2021) FAIRness along the machine learning lifecycle using Dataverse in combination with MLflow. *ICDE Workshops*.
23. Pandit, H.J. and Esteves, B. (in press) Enhancing Data Use Ontology (DUO) for Health-Data Sharing by Extending it with ODRL and DPV. *Semantic Web*.
24. Franklin, M., Halevy, A. and Maier, D. (2005) From Databases to Dataspaces: A New Abstraction for Information Management. *SIGMOD Record*, 34(4):27–33.
25. Otto, B. et al. (2019) Reference architecture model for international data spaces. *International Journal of Information Management*, 45:182–199.
26. Cappiello, C., Gal, A., Jarke, M., Rehof, J. and Yang, Y. (2022) Data spaces: Design, deployment, and future directions. *ACM Computing Surveys*, 55(2):Article 46.
27. Werbrouck, J. et al. (2024) ConSolid: A federated ecosystem for heterogeneous multi-stakeholder projects. *Semantic Web*, 15:429–460.
28. Kurteva, A. et al. (in press) RePlanIT Ontology for Digital Product Passports of ICT: Laptops and Data Servers. *Semantic Web*.
29. Palma, R. et al. (in press) GloSIS: The Global Soil Information System Web Ontology. *Semantic Web*.
30. Baryannis, G., Jia, L. and Papadakis, E. (2025) FATO: The Food Allergen Traceability Ontology. *Semantic Web*.
31. Wu, J., Garcia, K., Mayer, S. and Albert, J. (2024) NutriLink: An Ontology for Linking Digital Receipts to Food Nutrition Information and Dietary Recommendations. *Semantic Web*.
32. Heuschkel, M., Höffner, K., Schmiedel, F., Labudde, D. and Uciteli, A. (2023) The ANthropological Notation Ontology (ANNO): A core ontology for annotating human bones and deriving phenotypes. *Semantic Web*.
33. Trypuz, R., Kulicki, P. and Sopek, M. (in press) Ontology of autonomous driving based on the SAE J3016 standard. *Semantic Web*.
34. Chari, S. et al. (in press) Explanation Ontology: A General-Purpose, Semantic Representation for Supporting User-Centered Explanations. *Semantic Web*.
35. Woods, C. et al. (in press) An ontology for maintenance activities and its application to data quality. *Semantic Web*.
36. Bareedu, Y.S. et al. (2024) Deriving semantic validation rules from industrial standards: An OPC UA study. *Semantic Web*, 15:517–554.
37. Ferranti, N. et al. (in press) Formalizing and Validating Wikidata's Property Constraints using SHACL and SPARQL. *Semantic Web*.
38. Robaldo, L. and Batsakis, S. (in press) On the interplay between validation and inference in SHACL. *Semantic Web*.
39. Penteado, B.E., Maldonado, J.C. and Isotani, S. (2023) Methodologies for publishing linked open government data on the Web. *Semantic Web*, 14:585–610.
40. Poveda-Villalón, M., Fernández-Izquierdo, A., Fernández-López, M. and García-Castro, R. (2022) LOT: An industrial oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence*, 111:104755.
41. Grüninger, M. and Fox, M.S. (1995) Methodology for the design and evaluation of ontologies. *IJCAI Workshop on Basic Ontological Issues in Knowledge Sharing*.
42. Garijo, D. (2017) WIDOCO: A Wizard for Documenting Ontologies. *ISWC Posters and Demonstrations*.
43. Gruber, T.R. (1995) Toward principles for the design of ontologies used for knowledge sharing. *International Journal of Human-Computer Studies*, 43(5–6):907–928.
44. Brickley, D. and Miller, L. (2014) *FOAF Vocabulary Specification 0.99*. Namespace document.
45. SPDX Working Group (2022) *SPDX Specification*. Version 2.3.

46. Miles, A. and Bechhofer, S. (eds.) (2009) *SKOS Simple Knowledge Organization System Reference*. W3C Recommendation.
47. Wilkinson, M.D. et al. (2016) The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3:160018.
48. Brank, J., Grobelnik, M. and Mladenić, D. (2005) A survey of ontology evaluation techniques. *Conference on Data Mining and Data Warehouses*.
49. Knublauch, H. and Kontokostas, D. (eds.) (2017) *Shapes Constraint Language (SHACL)*. W3C Recommendation.
50. Poveda-Villalón, M., Gómez-Pérez, A. and Suárez-Figueroa, M.C. (2014) OOPS! (OntOlogy Pitfall Scanner!). *International Journal on Semantic Web and Information Systems*, 10(2):7–34.
51. Avsec, Z. et al. (2019) The Kipoi repository accelerates community exchange and reuse of predictive models for genomics. *Nature Biotechnology*, 37.
52. Baylor, D. et al. (2017) TFX: A TensorFlow-based production-scale machine learning platform. *KDD*.
53. Guarino, N. and Welty, C.A. (2002) Evaluating ontological decisions with OntoClean. *Communications of the ACM*, 45(2):61–65.

Complementary families and future profiles

The main discussion summarises the boundary between the DAIMO core and specialised profiles. The table below preserves the detail of that scoping decision: what the current profile covers, what remains outside the core, and what should be addressed as a future extension without modifying the fourteen native classes.

Competency questions

R — Registration and publication (5)

1. CQ-R1: Which AI models has a given provider published as governed catalogue assets?
2. CQ-R2: For a given model, which catalogue offering registered it, by which provider, and when? (requires `subPropertyOf` chain)
3. CQ-R3: Licence and usage policy of a model.
4. CQ-R4: I/O contract for the invocation interface of a published model.
5. CQ-R5: For each offering, what concrete `ParticipantRole` subclass does the providing agent hold? (requires `subClassOf+`)

D — Discovery and selection (4)

1. CQ-D1: Which models solve a given task in a given application domain?
2. CQ-D2: Which of those models are usable under a non-commercial policy pattern?
3. CQ-D3: Which models expose a service endpoint, and what authentication method does each require?
4. CQ-D4: Models reaching a minimum metric threshold under a shared evaluation context.

E — Execution and auditability (5)

1. CQ-E1: For a given catalogue offering, what service endpoint, authentication, and I/O contract apply to invoking the underlying model? (requires entailment)
2. CQ-E2: For a given model deployment, which runs have been executed, by which agents, and under which execution authorisation?
3. CQ-E3: Implementation, algorithm, flow, and compute infrastructure used in a run.
4. CQ-E4: Audit evidence (hash, signer, timestamp) supporting a given run.
5. CQ-E5: Derived artefacts produced by a run, and under which authorisation.

V — Evaluation and reproducibility (5)

1. CQ-V1: Shared evaluation context of a given evaluation.
2. CQ-V2: Under a shared evaluation context, highest-scoring model for a metric.
3. CQ-V3: Ranking of models under the same evaluation context and metric.
4. CQ-V4: For a given model, which benchmark suites has it been evaluated on?
5. CQ-V5: Reproducibility artefacts backing an evaluation.

G — Governance bridge (4, DAIMO-specific)

1. CQ-G1: Offerings that include a given model.
2. CQ-G2: Deployments serving a model, with infrastructure and I/O contract.
3. CQ-G3: Authorisation (and agreement) that authorised a specific run.
4. CQ-G4: Full provenance bundle for a derived artefact across participant contexts.

SHACL shapes

The DAIMO SHACL layer contains nine structural nodal shapes, three conformance shapes over reused classes, and six cross-class SHACL-SPARQL invariants. The module `shapes/daimo-shapes.ttl` is itself declared as an independent `owl:Ontology` with its own `owl:versionIRI` under `https://w3id.org/pionera/daimo/shapes/`.

- **Completeness shapes (9)**: `AIAssetOfferingShape`, `ParticipantRoleShape`, `ModelDeploymentShape`, `IOContractShape`, `ExecutionAuthorizationShape`, `DerivedArtifactShape`, `SharedEvaluationContextShape`, `AuditEvidenceShape`, `CrossParticipantProvenanceRecordShape`.

Table 14. Complementary families and controlled evolution path.

Family	Coverage in current DAIMO	Controlled extension path
Resource availability	Endpoint through `ModelDeployment`, `dcat:DataService`, and `dcat:endpointURL`; policy and licence through ODRL and `dct:license`; downloadable distributions through DCAT when the model is published as a file; provider through DAIMO roles and participant context. `dct:publisher` is reserved for the registry or catalogue maintainer when applicable.	Keep in the core only the availability needed for discovery and invocation. Artefact formats such as ONNX or PyTorch are expressed with `dct:format/dcat:Distribution` or with `IOContract` when they affect invocation.
Technical reproducibility	Training and evaluation datasets through MLDCAT-AP, DCAT, and PROV-O; reproducibility artefacts through `it6:Flow`, evidence, checksum, and shared evaluation context.	Hyperparameters, software dependencies, framework versions, GPU, and memory belong to `daimo-reproducibility` or to more detailed reuse of ML-Schema/MEX/MLDCAT-AP, not to new core classes.
Model versioning and lineage	DAIMO can represent derivation and revision through PROV-O when the graph declares it, and can audit the execution–artefact–authorisation chain.	A lineage profile may add SHACL rules for `prov:wasDerivedFrom`, `prov:wasRevisionOf`, obsolescence status, and retraining traces without altering the fourteen native classes.
MLOps automation	Outside the core. DAIMO defines the semantic contract that a tool may populate, but does not implement connectors or automatic extraction.	Integrators from Hugging Face, MLflow, or model registries may populate DAIMO graphs and serve as subsequent applied validation.
Monitoring and drift	Outside the core unless results are published as contextualised evaluations.	A `daimo-monitoring` profile may model drift, degradation, observed latency, computational footprint, or sustainability as temporal metrics or evaluation events.
Security and compliance	The core covers policy, authorisation, audit evidence, checksum, signer, and timestamp. It does not model vulnerabilities, attacks, or automatic legal interpretation.	`daimo-security` and `daimo-compliance` profiles may add vulnerabilities, signatures, AI Act controls, bias/fairness/explainability, and domain-specific SHACL/ODRL rules.

- **Conformance shapes over reused classes (3)**: `OfferInDAIMOShape` requires `odrl:assigner` and `odrl:target` on every `odrl:Offer`; `MachineLearningModelInDAIMOShape` requires policy, title, and identifier on `it6:MachineLearningModel`; `RunInDAIMOShape` requires flow, algorithm, associated agent, and timestamp on `it6:Run`.

- **Cross-class invariants (6)**: `DerivationAuthorizationInvariant` (INV-1), `RunAgentAuthorizationInvariant` (INV-2), `DeploymentServiceInvariant` (INV-3), `AuthorizationTemporalInvariant` (INV-4), `OfferingPolicyTargetInvariant` (INV-5), and `OfferingAssignerInvariant` (INV-6); all encoded as `sh:SPARQLConstraint`.

SPARQL queries

The twenty-three SPARQL queries corresponding to the competency questions in Appendix are released in `daimo/queries/queries.md`. The validator runs each query over the OWL-RL closure of the ontology-plus-example graph and verifies that each returns at least one row. Exact counts appear in Table 11. CQ-G2 returns two rows—one per endpoint of the multi-protocol REST and gRPC deployment—confirming that modelling several services per deployment is a legitimate pattern directly observable from SPARQL.

Detailed traceability is kept in the appendix so as not to overload the evaluation section. The table below allows inspection of which classes and properties each CQ exercises and preserves in the PDF the minimum evidence needed to reproduce the interpretation of the row counts.

Glossary of abbreviations

The final glossary collects abbreviations that appear in several sections of the article. It is kept as a lookup table to avoid repeated definitions in the main body and to stabilise the terminology used in Spanish and technical English.

URI conventions

DAIMO uses the canonical namespace `https://w3id.org/pionera/daimo`, with term identifiers of the form `https://w3id.org/pionera/daimo#<Term>` and versioned IRIs under `https://w3id.org/pionera/daimo/<version>`, linked to each other through `owl:priorVersion`. The satellite modules are independent ontologies with their own versioned IRIs: the SHACL layer under `https://w3id.org/pionera/daimo/shapes/` and the alignment module under `https://w3id.org/pionera/daimo/align` (the latter declaring `owl:imports` on the base ontology). The example graph uses the local namespace `https://example.org/daimo-scenario/`, clearly separated from the canonical space to avoid confusion between demonstration identifiers and ontology terms. Content negotiation is designed to resolve the same identifier to HTML, Turtle, RDF/XML, JSON-LD, or N-Triples depending on the `Accept` header. Until the redirect is formally registered, the source repository and the archived release provide the authoritative access points for review and replication.

Hash IRIs are used only for vocabulary terms, not for high-volume operational instances. In a production dataspace, concrete models, datasets, runs, and derived artefacts should be identified under participant or federated-catalogue namespaces; DAIMO provides the classes and predicates used to describe those resources. Native terms use English ASCII identifiers: singular *UpperCamelCase* for classes (`AIAssetOffering`, `ModelDeployment`), verbal or

Table 15. CQ–class–property–result traceability.

CQ	Summarised question	Main classes	Properties traversed	Rows
R1	Models published by a provider	`AIAssetOffering`, `MachineLearningModel`	`offeredBy`, `offersModel`	3
R2	Offering, provider, and date for a model	`AIAssetOffering`, `CatalogRecord`	`offersModel` ⊑ `foaf:primaryTopic`, `offeredBy`, `dct:issued`	1
R3	Applicable licence and policy	`MachineLearningModel`, `odrl:Offer`	`hasOfferPolicy`, `dct:license`, `odrl:permission`	1
R4	Input/output contract	`ModelDeployment`, `IOContract`	`hasIOContract`, `inputFormat`, `outputFormat`, `authMethod`	2
R5	Concrete provider role	`ParticipantRole`, role subclasses	`offeredBy`, `rdf:type/rdfs:subClassOf*`	2
D1	Models by task and domain	`MachineLearningModel`, `Task`	`it6:hasTask`, `dcat:theme`	3
D2	Models by policy pattern	`AIAssetOffering`, `odrl:Offer`	`hasOfferPolicy`, `odrl:permission`, `odrl:prohibition`	2
D3	Endpoints and authentication	`ModelDeployment`, `DataService`, `IOContract`	`exposedAs`, `endpointURL`, `authMethod`	4
D4	Minimum metric threshold	`Evaluation`, `SharedEvaluationContext`	`usesEvaluationContext`, `it6:hasEvaluationMeasure`, `it6:hasValue`	1
E1	Endpoint from an offering	`AIAssetOffering`, `ModelDeployment`, `IOContract`	`offersModel`, `hasDeployment`, `exposedAs`, `hasIOContract`	4
E2	Authorised runs	`ExecutionAuthorization`, `Run`	`authorizesRun`, `grantedTo`, `prov:wasAssociatedWith`	1
E3	Implementation and infrastructure	`Run`, `Flow`, `ComputerInfrastructure`	`it6:hasFlow`, `it6:hasAlgorithm`, `onInfrastructure`	2
E4	Audit evidence	`AuditEvidence`, `spdx:Checksum`	`evidenceOf`, `integrityHash`, `signedBy`, `recordedAt`	1
E5	Derived artefacts	`DerivedArtifact`, `ExecutionAuthorization`	`derivedFromRun`, `underAuthorization`	1
V1	Shared context	`SharedEvaluationContext`, `Evaluation`	`contextTask`, `contextDataset`, `protocol`, `randomSeed`	1
V2	Best model under context	`Evaluation`, `MachineLearningModel`	`usesEvaluationContext`, `it6:hasEvaluationMeasure`, `it6:hasValue`, `ORDER BY`	1
V3	Ranking under same metric	`Evaluation`, `SharedEvaluationContext`, `MachineLearningModel`	`usesEvaluationContext`, `it6:evaluates`, `it6:hasEvaluationMeasure`, `it6:hasValue`, `ORDER BY`	2
V4	Model benchmarks	`MachineLearningModel`, `Benchmark`	`it6:hasBenchmark`, optional `dct:title`	1
V5	Reproducibility artefacts	`Flow`, `AuditEvidence`, `DerivedArtifact`	`records`, `evidenceOf`, `integrityHash`	4
G1	Offerings including a model	`AIAssetOffering`, `MachineLearningModel`	`offersModel`, `hasOfferPolicy`	1
G2	Deployments per model	`ModelDeployment`, `IOContract`	`deploysModel`, `onInfrastructure`, `hasIOContract`	2
G3	Agreement that authorised a run	`ExecutionAuthorization`, `Run`, `odrl:Agreement`	`authorizesRun`, `grantedTo`, `expiresAt`	1
G4	Full provenance bundle	`CrossParticipant ProvenanceRecord`, `DerivedArtifact`	`records`, `spansParticipantContext`, `GROUP_CONCAT`	1

relational *lowerCamelCase* for properties (`offersModel`, `authorizesRun`, `usesEvaluationContext`), and slash segments for modules and versions. Translations, synonyms, and explanatory text are expressed with `rdfs:label`, `rdfs:comment`, and SKOS, not through URI variants.

Table 16. Abbreviations used in this article.

Abbrev.	Expansion
AI Act	Regulation (EU) 2024/1689 on artificial intelligence
BFO	Basic Formal Ontology
CQ	Competency Question
DAIMO	Dataspace AI Model Ontology
DCAT	Data Catalog Vocabulary
DCAT-AP	DCAT Application Profile
DL	Description Logic
DOI	Digital Object Identifier
DPV	Data Privacy Vocabulary
DSP	Dataspace Protocol
EDC	Eclipse Dataspace Components
FAIR	Findable, Accessible, Interoperable, Reusable
FOAF	Friend Of A Friend vocabulary
GFO	General Formal Ontology
I/O	Input / Output
LOT	Linked Open Terms methodology
MLDCAT-AP	Machine Learning Data Catalogue Application Profile
OOPS!	OntOlogy Pitfall Scanner
ODRL	Open Digital Rights Language
OWL	Web Ontology Language
OWL-RL	OWL 2 RL profile / rule-based reasoning
PROV-O	PROV Ontology
RDF	Resource Description Framework
RDFS	RDF Schema
REST	Representational State Transfer
SHACL	Shapes Constraint Language
SKOS	Simple Knowledge Organization System
SPARQL	SPARQL Protocol and RDF Query Language
SPDX	Software Package Data Exchange
TTL	Turtle (RDF serialisation)
UFO	Unified Foundational Ontology
WIDOCO	Wizard for Documenting Ontologies